Executable Archaeology: Reanimating the Logic Theorist from its IPL-V Source

Jeff Shrager, Ph.D.
Bennu Climate, Inc.
jeff@shrager.org

***Abstract*—The Logic Theorist (LT), created by Allen Newell, J.C. Shaw, and Herbert Simon in 1955–1956, is widely regarded as the first artificial intelligence program. While the original conceptual model was described in 1956, it underwent several iterations as the underlying Information Processing Language (IPL) evolved. Here I describe the construction of a new IPL-V interpreter, written in Common Lisp, and the faithful reanimation of the Logic Theorist from code transcribed directly from Stefferud's 1963 RAND technical report. Stefferud's version represents a pedagogical re-coding of the original heuristic logic into the standardized IPL-V. The reanimated LT successfully proves 16 of 23 attempted theorems from Chapter 2 of *Principia Mathematica*, results that are historically consistent with the original system's behavior within its search limits. To the author's knowledge, this is the first successful execution of the original Logic Theorist code in over half a century.**

***Index Terms*—Logic Theorist, IPL-V, artificial intelligence history, software archaeology, Newell, Simon, Shaw, list processing, digital preservation**

I. Introduction

In the summer of 1956, a small group of researchers gathered at Dartmouth College for a workshop on what John McCarthy had dubbed "artificial intelligence." Among them were Allen Newell and Herbert Simon, who arrived with something none of the other attendees had: a working AI program. While the other participants came with proposals and ideas, Newell and Simon brought a system that actually ran—a program that could discover proofs of mathematical theorems.[1]

The Logic Theorist (LT) [1] would ultimately prove 38 of the first 52 theorems in Chapter 2 of Whitehead and Russell's *Principia Mathematica* [3], in some cases finding proofs more elegant than those produced by the human authors. For Theorem 2.85, LT discovered a shorter, more direct proof than the one published in *Principia*. Simon showed the new proof to Bertrand Russell himself, who "responded with delight" [9, p. 167]. Edward Feigenbaum, then an undergraduate taking a graduate course from Simon at GSIA, recalls the moment the project's significance was first announced. As he later told Pamela McCorduck: "It was just after Christmas vacation—January 1956—when Herb Simon came into the classroom and said, 'Over Christmas Allen Newell and I invented a thinking machine.' And we all looked blank" [9, p. 138].

Beyond reproducing *Principia* proofs, the Logic Theorist introduced heuristic search strategies—means–ends analysis and subgoal decomposition—derived from protocol analyses of human problem solving. These ideas later became foundational in both artificial intelligence and cognitive architectures such as the General Problem Solver (GPS) [13], Soar, ACT, and EPAM [14]. LT thus sits at the very beginning of the fusion of AI and cognitive psychology.

LT was implemented in the Information Processing Language (IPL), a language created by the same team—Newell, Shaw, and Simon—specifically for building cognitive models [**?**]. IPL introduced list manipulation, dynamic memory allocation, symbols as first-class objects, recursion, and higher-order functions. It was explicitly a direct ancestor of Lisp [5]—McCarthy himself acknowledged IPL's influence. Yet while Lisp has thrived for nearly seven decades, IPL has been extinct since the mid-1960s; While numerous IPL-V implementations existed in the 1960s on various machines, no maintained implementations appear to be available in modern environments. The only modern attempt known to the author is Richman's unpublished IPL-V interpreter, written in Microsoft BASIC, which he used to run EPAM IV in the early 1990s [15].[2]

This paper describes the construction of a complete IPL-V interpreter in Common Lisp, and the faithful reanimation of the Logic Theorist from code transcribed directly from Stefferud's 1963 RAND technical report [6]. More than 99% of the LT code running in this reconstruction is the original code from that report. The reanimated LT proves 16 of 23 attempted theorems from *Principia Mathematica*, with results that are historically consistent with the original system's known behavior.

This work illustrates a broader methodological approach that might be called *executable archaeology*: the study of historically significant computational systems through faithful reconstruction and execution of their original code. Early AI programs were not merely theoretical proposals but functioning machines whose behavior often cannot be fully understood from published descriptions alone. Reanimating such systems provides a new empirical window into the origins of artificial intelligence.

[1]It is worth noting that Art Samuel's checkers-playing program existed a couple of years before LT, and it too was heuristic. But Samuel's goal was to build a checkers player, whereas Newell, Simon, and Shaw's explicit goal was to model human cognition across many domains. Samuel's effort was brilliant in its own right—he also built arguably the first machine learning program, and the first system to learn from self-play.

[2]Richman wrote to the author (personal communication, Sep 26, 2025, quoted with permission): "I just interpreted IPL-V in Microsoft Basic using the IPL-V manual that I found at the CMU library."

In prior work, a team including the present author restored Weizenbaum's original ELIZA to operation on a reconstructed CTSS running on an emulated IBM 7094 [7]. Although the Logic Theorist and ELIZA addressed very different problems—formal theorem proving and natural-language conversation—both programs emerged from the same symbolic programming lineage originating in IPL and later realized in Lisp and related list-processing systems. Each can be understood as a rule-based control system operating over symbolic list structures.

The present project differs from the ELIZA reanimation in both method and character: where the ELIZA work reconstructed an entire hardware and operating system stack, the LT reanimation emulates the abstract IPL-V machine in its direct descendant language, Common Lisp. And where ELIZA was a team effort, the LT project was largely a solo endeavor—albeit one in which large language models played an unexpectedly essential role at a critical juncture, as described in Section VI.

II. Historical Background

A. The Origins and Versions of the Logic Theorist

The story of the Logic Theorist begins in the early 1950s at the RAND Corporation. Herbert Simon and Allen Newell, collaborating with J.C. (Cliff) Shaw, sought to simulate complex thought by manipulating symbols. However, the history of the program is characterized by a series of shifting specifications across different versions of the Information Processing Language.

The system described in the famous 1956 RAND report (P-868) was a "paper" version, often referred to as IPL-I or the "Logic Language" (LL). At the time of the Dartmouth workshop in the summer of 1956, the program was not yet running on a machine; Newell and Simon noted it was "essentially specified" and that hand simulations suggested it would prove the 60-odd theorems of Chapter 2 of *Principia Mathematica*. Before the program ran on any machine, Newell and Simon famously simulated parts of it by hand using "the human computer"—family members and students acting as program components.

It was not until the implementation of IPL-II on the JOHNNIAC in late 1956 and early 1957 that machine-executable proofs were generated. The widely cited result that LT proved 38 of the first 52 theorems in *Principia*—including the discovery of a more elegant proof for Theorem 2.85 than the one produced by Whitehead and Russell—stems from these 1957 runs.

The code used in this reanimation, transcribed from Stefferud's 1963 memorandum, represents a third distinct phase. Stefferud's version was a pedagogical re-coding of the original heuristic logic into IPL-V, the first version of the language made widely available for public use. Thus, while our reanimation is a faithful execution of the original code published by the RAND team, it is a restoration of the 1963 "modernized" version of the 1956 conceptual model.

B. IPL: The Invention of Symbol- and List-Processing

To implement LT, Newell, Shaw, and Simon needed a programming language suited to symbolic reasoning. No such language existed, so they created the "Information Processing Language." As noted, IPL went through several iterations. While the earliest conceptual versions (IPL-I and IPL-II) powered the 1956 simulations and 1957 JOHNNIAC runs, IPL-V, released in 1964, was the version that became the definitive public standard [4].

TABLE I
Evolution of the Logic Theorist and IPL

Year	Version	Platform	Historical Milestones
1956	IPL-I (LL)	Manual	Conceptual spec; Hand-simulated only.
1957	IPL-II	JOHNNIAC	Proved 38 of 52 *Principia* theorems.
1958	IPL-IV	IBM 704	Internal RAND use; transitioning to IPL-V.
1963	IPL-V	Various	Stefferud's pedagogical LT implementation.
2025	IPL-V	Lisp/LLM	Reanimation using Stefferud's source code.

IPL-V runs on an abstract machine organized around *cells*... Each cell has an associated symbol, and any cell can be pushed onto or popped from any other cell acting as a stack head—there is no single distinguished stack. Symbols are central to IPL-V: they are first-class objects, and every data structure, routine, and control element is a named, addressable cell. A set of control cells (H0–H5) govern execution: H0 serves as the parameter and result cell, H1 as the program counter with its associated call stack, H2 as the free list, and H3 as a cycle counter. Ten working cells (W0–W9) provide scratch storage. Everything in IPL-V—data, routines, even the program itself—is a list of symbols. Routines are invoked by executing their name as an instruction; the machine traverses the list associated with that name, interpreting each cell as an instruction.

The language's built-in operations are called J-functions (because their names begin with "J": J0, J1, J2, and so on). There are approximately 150 of these, implementing operations from basic list manipulation to generator protocols (a form of lazy iteration that anticipates modern iterators). In the original IPL-V, many J-functions were themselves written in IPL, although some required machine-level primitives.

IPL was explicitly the direct ancestor of Lisp [5]—McCarthy acknowledged this lineage. It introduced list and symbol manipulation, the idea that programs are themselves lists, dynamic memory allocation, recursion, and higher-order functions. Having now spent months implementing an IPL-V interpreter, I can report that IPL had essentially *everything* that Lisp has, with two critical exceptions: homoiconic syntax and automatic garbage collection. Where Lisp unified code and data in a single, elegant, parenthesized notation, IPL expressed its ideas in an assembly-language style that, while powerful, was far more difficult to read and write. And where Lisp introduced garbage collection to automate memory reclamation, IPL required explicit management of the free list.

It is worth noting that IPL-V programs could, in principle, modify their own code at runtime—the same self-modification capability for which Lisp is celebrated. Because IPL-V routines are simply lists, and lists can be manipulated by IPL-V programs, nothing prevents a program from rewriting its own procedures. In practice this would be far more cumbersome than in Lisp, precisely because of the lack of homoiconic

syntax, but the capability was there. McCarthy's genius was not in inventing list processing *de novo*, but in finding the syntactic and runtime forms that made these ideas—ideas that Newell, Simon, and Shaw had already developed—elegant, accessible, and practical. Having programmed in Lisp for fifty years and attributed its brilliance entirely to McCarthy, building this interpreter gave me a deepened appreciation for Newell, Simon, and Shaw's foundational contributions.

III. Source Materials and Transcription

Unlike the ELIZA reanimation, which began with the dramatic discovery of original source code printouts in Weizenbaum's archives at MIT [7], the source materials for the Logic Theorist were already publicly available, if not widely known. Two documents were indispensable:

1) **Stefferud (1963)**, *The Logic Theory Machine: A Model Heuristic Program*, RAND Corporation memorandum RM-3731 [6]. This report contains a complete listing of the LT program in IPL-V, along with detailed descriptions of its M-routines (the main heuristic procedures).
2) **Newell et al. (1964)**, *Information Processing Language-V Manual*, 2nd edition [4]. The definitive reference for IPL-V, including the semantics of all J-functions, cell structure conventions, and the generator protocol.

Both documents proved to be exceptionally well-written. There were, however, occasional ambiguities—places where the specification assumed familiarity with conventions of 1960s computing (such as BCD character encoding) that are no longer common knowledge.

The LT code was transcribed from Stefferud's report into a Google spreadsheet by the author and Anthony Hay. Subsequently, Rupert Lane identified several transcription errors using his "gridlock" tool [11], software designed to accurately extract tabular code from PDF documents. Despite multiple people putting multiple sets of eyes on the code, a surprising number of small transcription errors persisted through several rounds of review.

The transcribed code was converted into an S-expression format (`.liplv` files) suitable for loading by the Lisp-based interpreter. Conveniently, Lisp comment characters work in these files, allowing corrections and annotations to be documented inline.

IV. A New IPL-V Interpreter (In Lisp!)

I implemented a new IPL-V interpreter to execute the Logic Theorist listing. The interpreter implements the symbolic data structures and execution model described in the IPL-V manual and supports the primitives required by the program. Although the earliest versions of the Logic Theorist ran on the RAND JOHNNIAC, by the time the Stefferud listing used here was published (1963) IPL-V had already been implemented on multiple machines. Accordingly, this reconstruction executes the program within a machine-independent IPL-V interpreter rather than attempting to reproduce a specific JOHNNIAC hardware environment. The full interpreter and transcription of the Logic Theorist code are available in the repository associated with this paper.

I chose to emulate the IPL-V abstract machine, rather than emulating the original hardware environment (as was done for ELIZA on the IBM 7094 and CTSS [7]), for several reasons. First, as mentioned just above, the code I am using is IPL-V, and so is not JOHNNIAC specific. Also, no thorough JOHNNIAC emultor exists, and in any case we do not have the JOHNNIAC IPL code. An IBM 7094 emulator by David Pitts includes an IPL-V binary interpreter running under a variant of the IBSYS batch operating system,[3]. Unfortunately, that interpreter is binary with no source code and no documentation. I ttempted to run our transcribed LT on this platform, but it failed early in execution, and because there is no source code it is essentially undebuggable.

More fundamentally, however, one of my central goals in undertaking this project was not merely to get LT to run, but to *elucidate* IPL-V—to develop a deep understanding of the language, the machine model, and the infrastructure of the period. Writing my own interpreter was essential to that goal. Second, IPL-V is defined as an abstract machine specification, not as a program for a specific computer; it was implemented on many different machines through the mid-1960s before being supplanted by Lisp. Third, there is a pleasing circularity in implementing IPL-V—Lisp's direct ancestor—in Lisp itself.

My interpreter (`iplv.lisp`) implements the complete IPL-V abstract machine as described in the 1964 manual: the cell table, the control cells H0–H5, the working cells W0–W9, and the push-down mechanisms. Lists are stored as linked chains of cells, exactly as specified. It is worth noting that the push-down (stack) mechanism is itself implemented as linked lists within the IPL-V system. Push and pop operations, which appear to be hardware primitives of the abstract machine, are internally built on the same list mechanism that underlies everything else in IPL-V, including data structures, routines, and the program itself. IPL-V is, in a very real sense, lists all the way down.

The core of my interpreter is a function, `ipl-eval`, which fetches and executes instructions by traversing the list named in the program counter (H1). `ipl-eval` is re-entrant, supporting the recursive subroutine calls that IPL-V requires. The J-functions are implemented as Lisp functions dispatched by name. While the interpreter implements the full IPL-V machine model, only the subset of J-functions required by LT (and a few test programs) is currently implemented, amounting to several dozen of the approximately 150 defined in the manual.

A complete run of LT—attempting proofs of 23 theorems—takes approximately 574,000 IPL machine cycles, creating over 576,000 symbols[4], and finishes in a few seconds on a typical modern laptop. On the JOHNNIAC at RAND in 1956, this likely would have required hours.

V. What the Logic Theorist Proves

The Logic Theorist attempts to prove theorems in propositional logic using the five axioms of *Principia Mathematica*, Chapter 2. The notation used by the original authors is given in Table II.

[3] Available at https://www.cozx.com/dpitts/ibm7090.html

[4] Yeah, I was surprised too. It's really just a coincidence!

TABLE II
LOGICAL NOTATION USED BY THE LOGIC THEORIST

Symbol	Meaning
I	Implication ($\rightarrow$)
V	Disjunction ($\vee$)
*	Conjunction ($\wedge$)
-	Negation ($\neg$)
=	Biconditional ($\leftrightarrow$)
.=.	Definitional equality

LT employs several proof methods:

- **Substitution**—substitute variables in an axiom or previously proved theorem to match the goal.
- **Detachment** (modus ponens)—from $A \rightarrow B$ and A, conclude B.
- **Forward chaining**—apply substitution and detachment working forward from known theorems.
- **Backward chaining**—work backward from the goal, seeking premises that would yield it.
- **Sublevel replacement**—replace a subexpression using a known equivalence.

Each proof attempt is bounded by an effort limit of 20,000 IPL machine cycles. This is a soft cap: it prevents the system from *beginning* a new subproblem exploration once exceeded, but cannot interrupt an in-progress search.

Table III shows the results of my reanimated LT on the 23 theorems attempted. Of these, 16 are proved and 7 are not—results that are historically consistent with the original LT's known behavior within its search effort limits.

TABLE III
THEOREMS ATTEMPTED BY THE REANIMATED LOGIC THEORIST

Thm.	Formula	Result
2.01	(PI-P)I-P	Proved
2.02	QI(PIQ)	Proved
2.04	(PI(QIR))I(QI(PIR))	Proved
2.05	(QIR)I((PIQ)I(PIR))	Proved
2.06	(PIQ)I((QIR)I(PIR))	Proved
2.07	PI(PVP)	Proved
2.08	PIP	Proved
2.10	-PVP	Proved
2.11	PV-P	Proved
2.12	PI--P	Proved
2.13	PV---P	Proved
2.14	--PIP	Not proved
2.15	(-PIQ)I(-QIP)	Not proved
2.20	PI(PVQ)	Proved
2.21	-PI(PIQ)	Not proved
2.24	PI(-PVQ)	Proved
3.13	(-(P*Q))I(-PV-Q)	Not proved
3.14	(-PV-Q)I(-(P*Q))	Not proved
3.24	-(P*-P)	Not proved
4.13	P=--P	Not proved
4.20	P=P	Proved
4.24	P=(P*P)	Proved
4.25	P=(PVP)	Proved

A representative proof trace is shown below. Here LT proves Theorem 2.01, (PI-P)I-P:

```
2.01     (PI-P)I-P

PROOF FOUND.

     GIVEN            *1.2   (AVA)IA
     SUBSTITUTION     .0     (PI-P)IP
     SUBLEVEL REPL    2.01   (PI-P)I-P
     Q.E.D.

EFFORT       LIMIT 20000    ACTUAL 5579
SUBPROBLEMS  LIMIT 50       ACTUAL 1
SUBSTITUTIONS LIMIT 50      ACTUAL 2
```

The proof begins from Axiom *1.2, (AVA)IA, substitutes -P for A to obtain (-PV-P)I-P, then applies sublevel replacement (using the fact that P implies PVP) to transform the antecedent, yielding the desired theorem.

VI. DEBUGGING: SOLO WORK, THEN A COLLABORATION WITH AI

A. The Solo Phase

The great majority of the development and debugging work on this project was done without LLM assistance. Building the interpreter, transcribing the code, understanding the IPL-V manual, and working through the first phase of bugs—crashes, unimplemented J-functions, stack underflows, register save/restore errors—was a solo effort spanning many months. These bugs were frustrating but tractable: a hard crash is almost always traceable to some error in the run-up to the crash, and a human programmer working alone can identify and fix them.

Over this period, I developed a strong sense that the IPL-V interpreter basically worked—otherwise LT would never have gotten through even a dozen instructions—and that the LT transcription was correct. Therefore, almost all remaining errors were likely in the J-functions, which I had written in Lisp from their descriptions in the Newell et al. manual. My experience was that the manual's J-function descriptions were sometimes vague in subtle ways, and this had bitten me before. I was particularly wary of the most complex J-functions: the generators (J17–J19, J100) and the list manipulation functions (J74 and others).

B. The Wall—and How LLMs Broke Through It

Unlike conventional programs where correctness can be verified stepwise, LT is a heuristic search program whose behavior emerges from many interacting procedures. Once the interpreter was stable enough that LT ran without crashing, the character of the bugs changed entirely. The output was now thousands of lines of execution trace—machine cycles, register states, list operations—that *might* be correct. LT was proving trivial theorems (those requiring substitution only), but it would not prove complex theorems that it should have, and would get into loops, failing to recognize when a proof had been found. The program was heuristic; it was supposed to explore and backtrack. Distinguishing a correct backtrack from one caused by a subtle bug required understanding both the IPL-V machine semantics and LT's high-level proof strategy simultaneously, while tracking state across tens of thousands of interpreted machine cycles.

This experience had an uncanny resonance with the original developers' accounts. In an interview conducted by Pamela

McCorduck for her pioneering history of AI [9]—transcripts originally obtained for the ELIZA reanimation project [7]—Cliff Shaw describes the particular difficulty of debugging programs that use dynamic storage allocation and heuristic search:

> "When we first used dynamic storage allocation and it kept the list of available space and dumped unused space back on this list… the memory was churning over in a way that none of us had ever experienced before. You take a memory dump and it was completely laced through with these links from these lists… [L]ater on, particularly in chess, it got so that the machine would produce behavior but you weren't at all sure that what you had made in the way of changes for this last run actually influenced that behavior in the way they were intended to. It would behave reasonably in some sense… but it was difficult to debug in a completely different sense. Not that it fell apart but that it wouldn't fall apart. You weren't sure that new programs, revisions, had the effect that you intended because it would do something reasonable—it wouldn't always do what you predicted and then you were in a quandary as to whether you had a bug or whether this was a consequence of what you had specified."
>
> —J. C. Shaw, conversation with P. McCorduck, June 16, 1975

Sixty years later, with the same family of programs, I faced the identical challenge. At this point progress had effectively stalled. The cognitive load of holding the machine state in my head across thousands of trace lines, while simultaneously questioning whether each line reflected correct or incorrect behavior, exceeded what I could sustain.

This is where I turned to large language models—specifically, Google's Gemini (via the AntiGravity interface) and Anthropic's Claude (via its terminal-based coding interface, Claude Code). They were not involved in the earlier phases of the project; they entered at the point where the nature of the debugging task shifted from crash analysis to the far more demanding problem of behavioral verification.

C. Teaching a Dead Language to an LLM

The collaboration began from a position of mutual ignorance. IPL-V is an essentially dead language: the manual is poorly OCR'd in places and obscure even where legible; the language itself is unlike any modern programming language; and there is no Stack Overflow, no GitHub corpus, no community of practitioners for the LLMs to have learned from. Few people remain who have direct experience programming in IPL-V. The LLMs' primary source of information about IPL-V was my own Lisp code—which, of course, still contained bugs.

This created a fascinating epistemological bootstrapping problem. The LLMs were inferring IPL-V semantics from my interpreter, sometimes correctly and sometimes not. When the interpreter had bugs, the LLMs would occasionally learn incorrect semantics and propagate those errors in their suggestions. Untangling this required constant vigilance about what was established knowledge versus what was inferred from possibly-buggy code. Programming with LLMs always involves a degree of project teaching, but in this case I also had to teach the language itself—a qualitatively different challenge.

One early difficulty was that an LLM would sometimes try to debug the LT program rather than my interpreter. We had to maintain the methodological assumption that the LT code was correct (even though we did not entirely know this), and focus debugging effort on the interpreter and especially the J-functions. Without this discipline, every anomaly would have opened two lines of investigation, making progress impossible.

D. The J74 Bug: Simon's J's to the Rescue

What the LLMs excelled at was tenacity. They were willing to read through thousands of lines of traces and dumps, insert breakpoints at hundreds of places, and maintain the kind of tedious but precise state-tracking that the task demanded—work that simultaneously required deep contextual understanding. This combination of exhaustive attention and domain comprehension is, ironically, precisely where current LLMs excel.

Gemini found the first significant issue after several hours of LLM time spread over several days, and as predicted it was in a J-function. This made substantial progress: now LT was generating proofs, but was not stopping when a proof was found. After literally another several hours of LLM time, Claude isolated the second problem in J74—the list copying function. But we did not know how to fix it, because both Claude and I agreed that my code appeared to match the manual's description. There was a clear problem, but the specification seemed to support my implementation.

This is where the project took its most interesting turn. We had access to "Simon's J's" from the Computer History Museum [10]—a set of original IPL-V punched cards from 1962. These cards represent the production implementation of the J-functions at the very moment IPL-V was being standardized for wider use. By this point I had taught Claude enough IPL-V that it was able to read the original 1962 code and compare it with my Lisp implementation of J74. The result: they differed in exactly the way we had hypothesized. A one-line patch to my J74 implementation made LT run completely correctly.

This was a remarkable moment: an AI system, having learned a dead programming language primarily from a modern interpreter that it had helped debug, was now able to read original 1962 source code and use it to identify and fix a bug. It was a collaboration across sixty years—Simon's original card deck, preserved by the Computer History Museum and transcribed by its volunteers, provided the ground truth that neither the manual's prose description, nor my interpretation of it, nor the LLM's inference from my code, had been able to supply.

E. Roles of the Collaborators

The human and LLM collaborators on this project played complementary but distinct roles. The human contributors—particularly Anthony Hay, Arthur Schwarz, Leigh Klotz, and David M. Berry—provided code transcription, error detection, and sustained intellectual encouragement through weekly progress updates. Rupert Lane's gridlock software caught transcription errors that had survived multiple rounds of human review. These contributions were essential and could not have been replaced by LLMs.

The LLMs, for their part, filled a role that no human collaborator was available to fill: the grinding, hour-after-hour work of reading thousands of lines of execution traces, inserting diagnostic instrumentation, and tracking machine state across tens of thousands of cycles to localize subtle behavioral bugs. Both Gemini and Claude made substantive debugging contributions. In the end, I settled on working primarily with Claude, not because of a significant capability difference, but because it was easier to maintain a single long session with accumulated context than to switch between systems. The complete interaction histories with both LLMs are preserved in the project repository.

VII. Reflections: IPL, Lisp, and the Arc of AI

I have been programming in Lisp for fifty years. For most of that time, I attributed the genius of Lisp entirely to John McCarthy. One of the most valuable outcomes of this project has been a revision of that understanding.

Building an IPL-V interpreter revealed that Newell, Simon, and Shaw had, by the late 1950s, already invented essentially every conceptual element that makes Lisp powerful: list manipulation, symbol processing, the idea that programs are themselves lists, dynamic memory allocation, recursion, higher-order functions, generators, and even the possibility of runtime self-modification. What they lacked were the all-important homoiconic syntax—the unification of code and data in a single parenthesized notation that makes Lisp programs manipulable by Lisp programs—and automatic garbage collection.

This is not to diminish McCarthy's contribution, which was genuinely brilliant. But it reframes the standard narrative. McCarthy's genius was not in inventing list processing *ex nihilo*, but in finding the syntactic and runtime forms that made these ideas—ideas that Newell, Simon, and Shaw had already developed—elegant and accessible. IPL-V and Lisp coexisted for approximately five years, but Lisp was so much simpler and more expressive that it rapidly supplanted its ancestor.

There is a personal dimension to this realization that bears on the nature of the project itself. I studied under both Newell and Simon at Carnegie Mellon University in the 1980s, where my work focused on modeling the cognition underlying scientific and engineering reasoning. IPL and LT were long since history by that time, and I never discussed either with them—we barely learned about these pioneering efforts in our classes. But the intellectual values they instilled—rigor about mechanism, respect for the complexity of cognition, and the conviction that thinking could be understood computationally—shaped the rest of my career. Now, at the end of that career, I find myself studying Newell, Simon, and Shaw *as* scientist-engineers: reconstructing their reasoning, reverse-engineering their design decisions, and trying to understand how they invented AI and cognitive science. The subject of my training—the cognition of scientists and engineers—and its object have folded into each other.

VIII. Related Work

This paper makes three contributions:

1) A working interpreter for IPL-V implemented in Common Lisp.
2) The successful execution of the Logic Theorist from Stefferud's IPL-V source listing, likely the first such execution in over fifty years.
3) A demonstration of executable reconstruction as a method for studying early AI systems.

The reanimation of historical software systems has gained momentum in recent years. Most directly related is the ELIZA reanimation by Lane et al. [7], in which a team including the present author restored Weizenbaum's original ELIZA to operation on a reconstructed CTSS running on an emulated IBM 7094. That project reconstructed the complete original hardware and operating system stack; the present work instead emulates the abstract machine specification in a modern language.

Reimplementing the Logic Theorist's *algorithms* in a modern language is a relatively straightforward exercise—essentially an introductory AI course project, and one that has surely been done many times. A 1968 paper, for example, describes a reimplementation in LISP 1.5 [12]. The present work is a fundamentally different undertaking: rather than rewriting LT's logic in a modern language, it constructs a faithful emulator of the IPL-V abstract machine and runs the original code, transcribed directly from a 1963 technical report. The distinction is significant: a Lisp reimplementation demonstrates that LT's algorithms can be expressed in Lisp (which is unsurprising), while the present work demonstrates that the original IPL-V code, as written by Newell, Simon, and Shaw's team, still functions correctly when provided with a faithful implementation of the machine it was designed for.

IX. Next Steps

The IPL-V interpreter developed for this project is not specific to the Logic Theorist. It implements the general IPL-V abstract machine and can, in principle, run any IPL-V program. I am in progress of building a corpus of early IPL-V AI programs. The most significant targets are the General Problem Solver (GPS) [13] and EPAM (Elementary Perceiver and Memorizer) [14]. EPAM source code is available in the Feigenbaum archives at Stanford, and archival work to locate complete source code for GPS is planned for spring 2026. Now that a proven IPL-V interpreter exists, running these programs should require substantially less effort than the initial construction of the interpreter itself.

The interpreter, the transcribed LT code, and the complete LLM interaction histories are available as open source at https://github.com/jeffshrager/IPL-V.

ACKNOWLEDGEMENTS

This paper was written with the aid of various LLMs, but all concepts and final editorial decisions were the author's alone.

The following people contributed to this project in ways ranging from code transcription to weekly encouragement: Anthony Hay, Arthur Schwarz, Leigh Klotz, David M. Berry, Rupert Lane, Amy Majczyk, Ethan Ableman, and Paul McJones. I am particularly grateful to Hay, Schwarz, Klotz, and Berry, who received weekly progress updates throughout the project and responded with encouragement that was always helpful and sometimes technically substantive. Rupert Lane's gridlock software caught several transcription errors that had survived multiple rounds of human review.

I also thank the Computer History Museum for preserving and making available "Simon's J's," the original 1962 IPL-V J-function card deck, which proved essential during debugging.

Finally, I want to acknowledge Allen Newell and Herbert Simon, under whom I studied at CMU in the 1980s. The subject of that training—the cognition of scientists and engineers—has, in this project, become inseparable from its object.

REFERENCES

[1] A. Newell, and H. A. Simon, *The Logic Theory Machine A Complex Information Processing System*, RAND Corp., Santa Monica, CA, USA, Rep. P-868, 1956.

[2] A. Newell, J. C. Shaw, and H. A. Simon, "Empirical explorations with the logic theory machine: A case study in heuristics," in *Proc. Western Joint Comput. Conf.*, 1957, pp. 218–230.

[3] A. N. Whitehead and B. Russell, *Principia Mathematica*. Cambridge, UK: Cambridge Univ. Press, 1910–1913.

[4] A. Newell, Ed., *Information Processing Language-V Manual*, 2nd ed. Englewood Cliffs, NJ, USA: Prentice-Hall, 1964.

[5] J. McCarthy, "Recursive functions of symbolic expressions and their computation by machine, Part I," *Commun. ACM*, vol. 3, no. 4, pp. 184–195, Apr. 1960.

[6] E. Stefferud, "The Logic Theory Machine: A Model Heuristic Program," RAND Corp., Santa Monica, CA, USA, Rep. RM-3731-CC, 1963.

[7] R. Lane, A. Hay, A. Schwarz, D. M. Berry, and J. Shrager, "ELIZA Reanimated: Restoring the mother of all chatbots to one of the world's first time-sharing systems," *IEEE Ann. Hist. Comput.*, vol. 47, no. 1, pp. 8–23, 2025.

[8] F. Gruenberger, "The History of the JOHNNIAC," RAND Corp., Santa Monica, CA, USA, Rep. RM-5654-PR, 1968.

[9] P. McCorduck, *Machines Who Think*, 2nd ed. Natick, MA, USA: A. K. Peters, 2004.

[10] A. Bochannek, "Simon's J's," Computer History Museum Blog, Sep. 26, 2012. [Online]. Available: https://computerhistory.org/blog/simons-js/ (Accessed: Mar. 11, 2026).

[11] R. Lane, "Gridlock: Accurate code extraction from PDFs," 2024. [Online]. Available: https://github.com/rupertl/gridlock

[12] J. S. Millstein, "The Logic Theorist in LISP," *Comput. J.*, vol. 11, no. 1, pp. 39–44, Feb. 1968.

[13] A. Newell, J. C. Shaw, and H. A. Simon, "Report on a general problem-solving program," in *Proc. Int. Conf. Inform. Process.*, 1959, pp. 256–264.

[14] E. A. Feigenbaum, "The simulation of verbal learning behavior," in *Proc. Western Joint Comput. Conf.*, 1961, pp. 121–132.

[15] H. B. Richman, J. J. Staszewski, and H. A. Simon, "Simulation of expert memory using EPAM IV," *Psychol. Rev.*, vol. 102, no. 2, pp. 305–330, 1995.